

AI Complaint Resolution Agent

Using RAG and Prompt Optimization

Mini Project – Review 1 | Idea Presentation + Literature Survey

Team Members: _____ Course / Faculty:

Introduction

- Customer complaints are often handled manually, requiring support staff to understand the complaint and search relevant policies.
- Manual handling can result in delayed responses, inconsistent resolutions, and difficulty locating the correct policy information.
- The proposed system combines Generative AI, Retrieval-Augmented Generation (RAG), and prompt engineering.
- The system will convert a natural-language complaint into a structured, policy-grounded resolution.



Problem Statement

To develop an AI-based complaint resolution system that:



Understands customer complaints



Identifies complaint category, sentiment and priority



Retrieves relevant company policies using RAG



Generates a clear response grounded in the retrieved policy

Objectives

1

Classify Complaints

Classify complaints into categories such as refund, return, delivery, damaged product, warranty and payment.

2

Detect Sentiment & Priority

Detect customer sentiment and assign complaint priority.

3

Build Knowledge Base

Create a small company-policy knowledge base for retrieval.

4

Vector Retrieval

Use embeddings and a vector database to retrieve relevant policy chunks.

5

Generate Resolution

Generate a structured resolution and customer-facing response using an LLM.

6

Compare & Refine Prompts

Compare prompting strategies and refine the prompt using evaluation results.

Existing System / Current Approach

Typical Manual Process



Challenges

Time-consuming for large complaint volumes

Policy lookup may be inconsistent

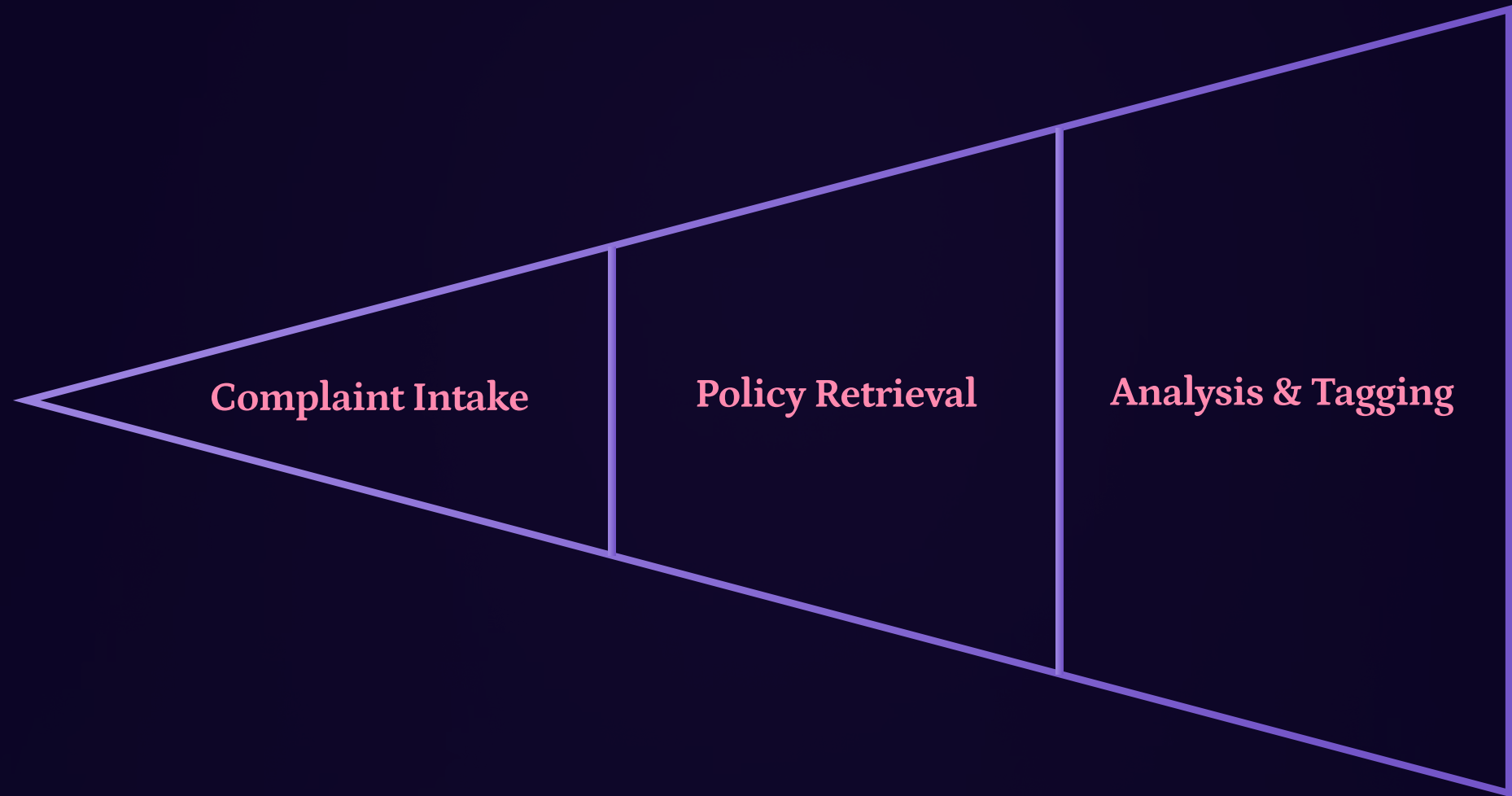
Responses can vary between agents

Harder to scale during high demand

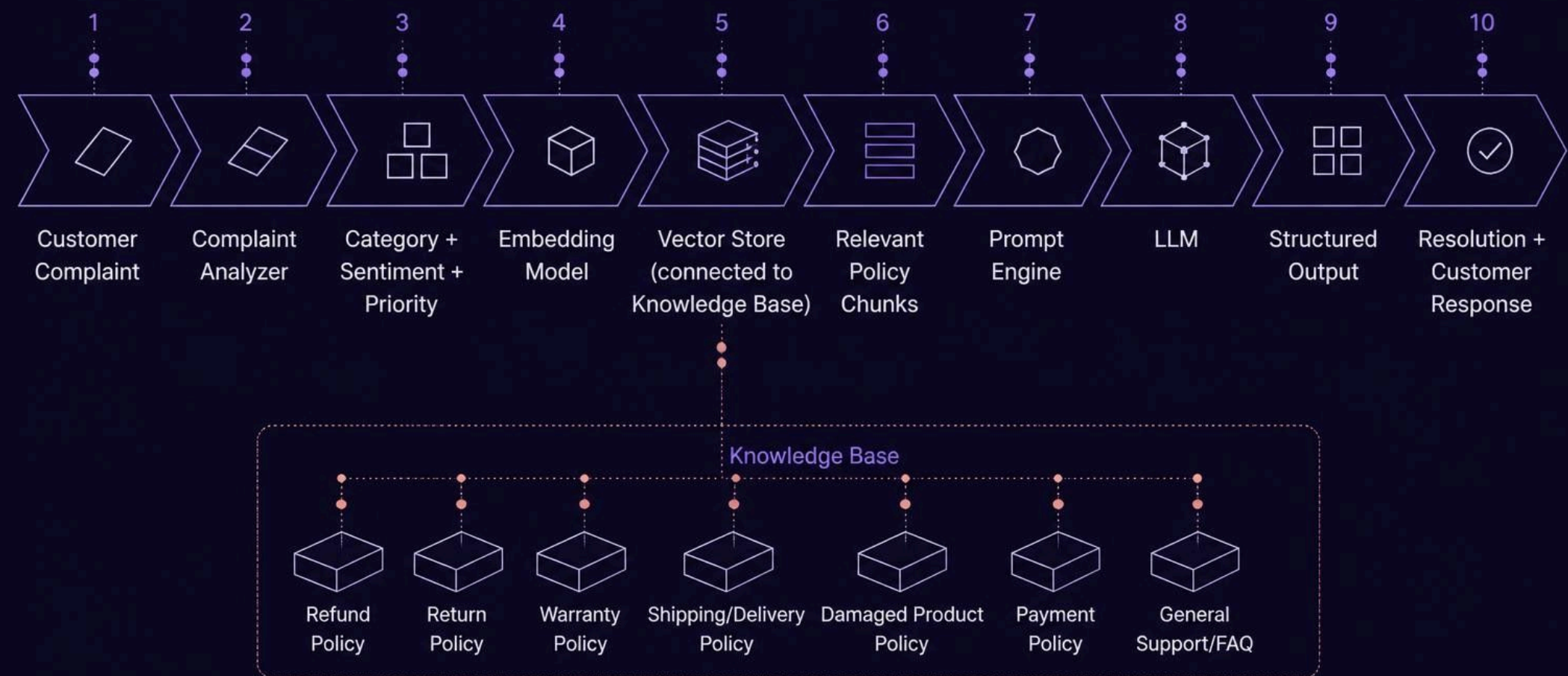
Knowledge is not automatically retrieved and grounded

Proposed System

The proposed system receives the complaint, identifies category, sentiment and priority, retrieves relevant policy chunks, and passes those chunks to an LLM through an optimized prompt.



High-Level Architecture



RAG Pipeline

01

Collect Policy Documents

Collect policy documents: refund, return, warranty, shipping, payment, damaged-product and support policies.

03

Convert to Embeddings

Convert chunks into vector embeddings.

05

Retrieve Relevant Chunks

Convert the complaint into an embedding and retrieve the most relevant chunks.

02

Split Documents

Split documents into smaller chunks.

04

Store in Vector Database

Store embeddings in FAISS or ChromaDB.

06

Ground LLM Response

Pass the retrieved policy context to the LLM so the response is grounded in the knowledge base.

Prompt Engineering & Optimization

Zero-shot Prompting

Direct task instruction.

Role Prompting

Define the model as a complaint-resolution specialist.

Few-shot Prompting

Provide representative examples.

Structured Prompting

Explicitly request category, sentiment, priority, resolution and response.

RAG Prompting

Instruct the model to use retrieved policy context.

Optimization

Compare outputs using relevance, clarity, consistency and policy adherence, then refine the prompt.

Literature Survey – RAG & Prompting

Work	Year	Main Contribution	Relevance to Our Project
Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks"	2020	Introduced RAG models combining parametric generation with a dense retrieval index.	Provides the core RAG idea for retrieving policy knowledge before generation.
Gao et al., "RAG for Large Language Models: A Survey"	2023	Surveys RAG architectures, retrieval, augmentation, generation and evaluation.	Helps select and explain the retrieval + generation pipeline.
Wei et al., "Chain-of-Thought Prompting..."	2022	Showed that prompting with reasoning examples can improve performance on reasoning tasks.	Motivates structured/few-shot prompt experiments; our project evaluates what is useful for complaint resolution.